Watching a robot do its job

A monitor that knows the stages of a task

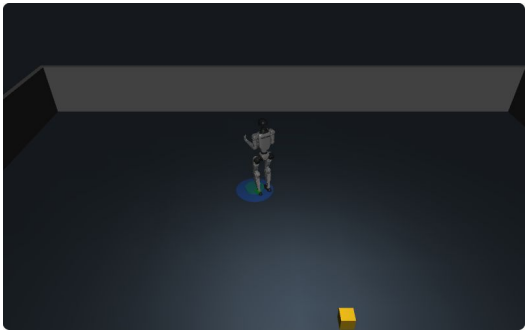
Pinhas Nehoray Aburmad

Ben-Gurion University of the Negev
Supervisor: Prof. Ronen Brafman



The problem: robots fail silently

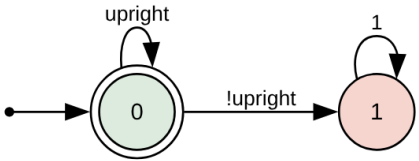
- A task has steps: walk, pick up, carry, put down.
- Things go wrong: the box slips, the grasp misses.
- Often the robot **does not know**, and reports “done”.
- We want an **independent watcher** that notices **at once**.



The robot reports success at the drop-off (blue). The box (yellow) is still on the floor.

Background: a runtime monitor

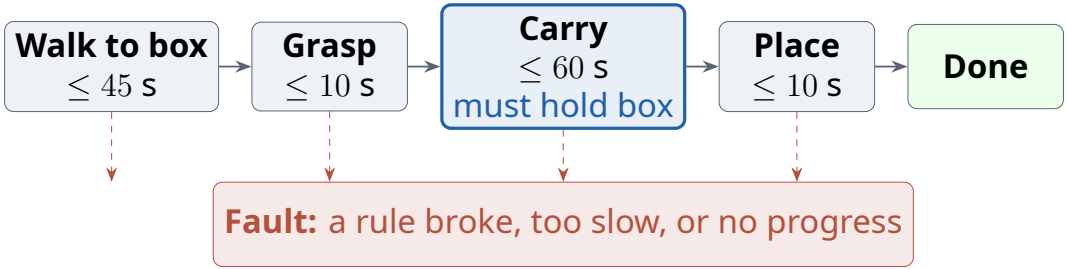
- A **separate program** that only reads the robot's messages.
- It checks **rules over time**, in **LTL**: "*always* upright", "*eventually* deliver".
- Each rule becomes a small **automaton**; one step per second.
- Catch: "eventually" **never fails early**.



"Always upright": one fall moves to state 1 (red) for good.

Our idea: give the monitor the stages

Each stage has its own rules: what stays true, when it ends, a time limit.



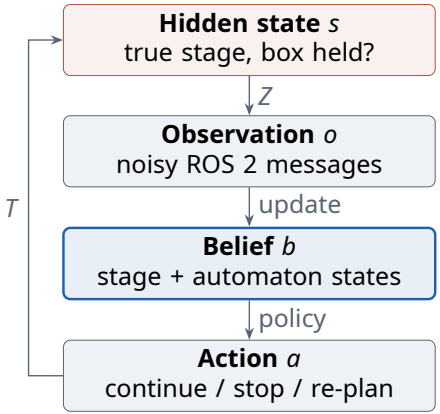
Example: *while carrying, the robot must hold the box*, so a drop breaks the rule **at once**.

The model behind it: a POMDP

Definition: a POMDP $\langle S, A, O, T, Z, R \rangle$; the agent keeps a belief b over hidden states:

$$b'(s') \;\propto\; Z(o \mid s') \sum_{s} T(s' \mid s, a)\, b(s)$$

- s includes the **true stage**; o = ROS 2 messages; a = continue / stop / re-plan.
- So one o means different things: “not holding” is fine while walking, a fault while carrying.



What our monitor computes today

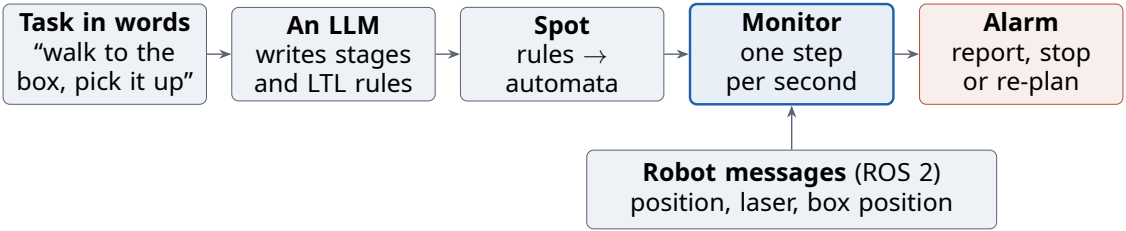
Claim: the monitor is a **deterministic belief filter** with a fixed policy.

1. Belief = one point: $b = (\text{stage}, q_1, \ldots, q_k)$; q_i = state of automaton i.
2. Update = one automaton step: $q_i' = \delta_i(q_i, \sigma(o))$.
3. Policy = fixed: a broken rule $\Rightarrow$ report, stop or re-plan.

Not yet: probabilities and an optimised policy. That is the next step.

$\sigma(o)$: true/false propositions computed from the messages, e.g. `holding_object`.

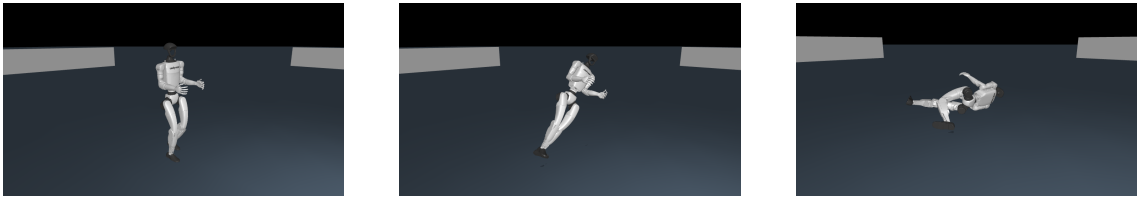
Method: how the monitor is built and run



- Rules generated **in one try**, checked automatically.
- Reads only sensors; **never asks the planner** how it is doing.

Test bed: a humanoid in a physics simulator

- Unitree **G1** in **MuJoCo**, with Unitree's own walking controller.
- **Faults injected on purpose**: push, stall, drop, failed grasp.
- Compare **with stages** vs. the same rules **without** ("flat").

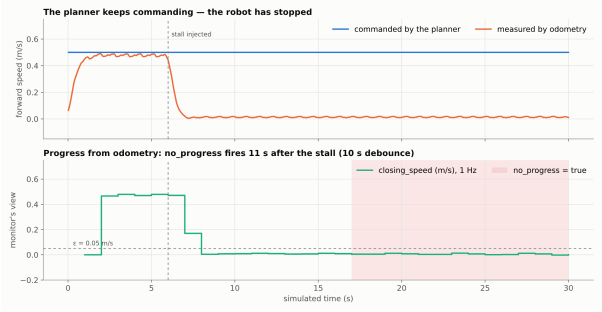


Walking → pushed → on the floor. Truth comes from the simulator, which the monitor never sees.

Experiment 1: a short walk (1800 runs)

- Two similar stages: they **added little** (15 vs. 14 of 15 stalls).
- What mattered: **progress from the robot's own sensors**. Without it: **1 of 15**.

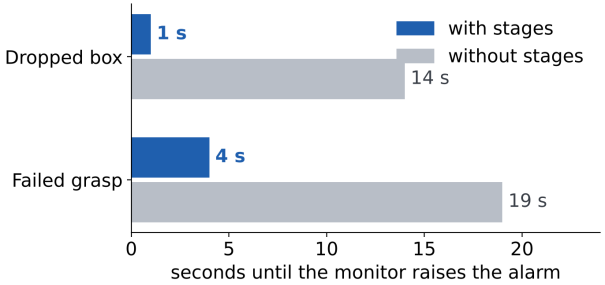
Lesson: don't trust the planner's report.



The planner commands 0.5 m/s; the robot has stopped. The monitor sees no progress.

Experiment 2: fetch and deliver (3360 runs)

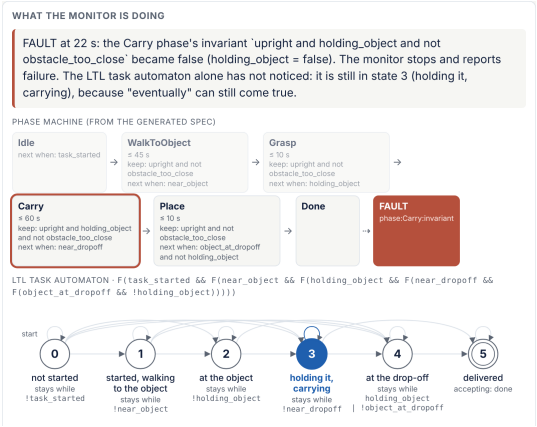
- Four stages, over ROS 2.
- Dropped box: **1 s** vs. 14 s.
- Failed grasp: **4 s** vs. 19 s.
- Earlier in **20 of 20** runs ($p = 1.9 \cdot 10^{-6}$).



Median over 20 runs of each fault.

Inside one run: the stage catches the drop

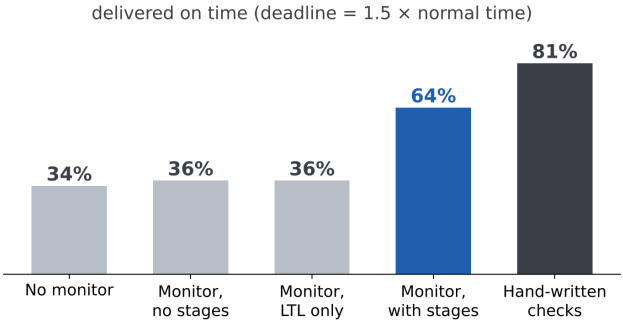
- **21.6 s:** the box drops.
- **22 s:** the Carry rule breaks: alarm.
- The LTL automaton is **still in state 3**: “eventually” can still happen.



One run, from the live demo: current stage and state lit.

Does early matter? Yes, with a deadline

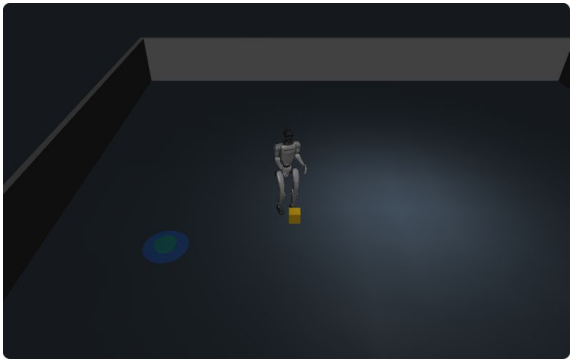
- No deadline: all monitors $\approx$ 82–84%.
- Deadline: **64%** with stages vs. 36%. LTL alone: 36%.
- Hand-written checks: 81%, but each fault must be **foreseen**.



160 runs per bar. Value of information, as the POMDP view predicts.

Faults nobody wrote a check for

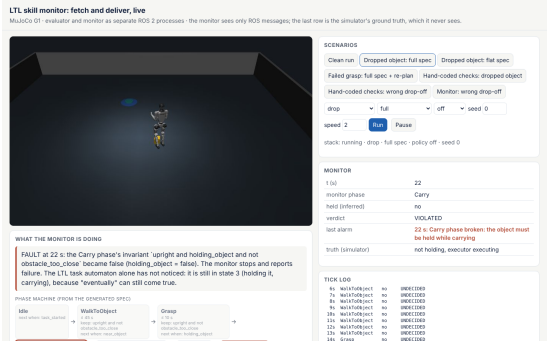
- Wrong drop-off place; passing too close to walls.
- Hand-written checks: **40 of 40** silent.
- With our monitor: **0 of 40** silent.
- Missed by all: slowness, detours.



The box is left far from the drop-off (blue); the robot reports success. The monitor flags it.

What we learned

- **Stages catch failures as they happen.**
- Early pays off **when time matters**.
- Generated from words; catches faults **nobody planned for**.
- Progress from the **robot's own sensors**.



Live demo, one command:

```
docker run -p 8080:8080
ghcr.io/pinhas019/ltlmonitor-demo
```

To be done



Limits today: simulated grasp (the G1 model has no arms), a stand-in for the camera, uncalibrated thresholds.